

TECHNICAL REPORT

Military Asset Management System

Architecture, data model, API, access control and setup guide

1. System overview
2. Overall architecture
3. ER schema diagram
4. API endpoint reference
5. RBAC authorization matrix
6. Setup steps
7. Sample test accounts

1 System overview

MAMS records how military equipment moves in and out of bases. Commanders and logistics staff can see opening balances, closing balances and net movement for any date range. Each base's stock is **calculated from its movement records** and is never stored as a counter. Four kinds of movement change the stock:

Movement	Effect on base stock	Table	Stock check
Purchase	Adds stock at the base	Purchase	No
Transfer	Removes stock from the source base and adds it to the destination base	Transfer	Yes, at the source base
Assignment	Removes stock (equipment issued to personnel)	Assignment	Yes
Expenditure	Removes stock (used up, destroyed or lost)	Expenditure	Yes

Balance formulas

```
Net Movement      = Purchases + Transfers In - Transfers Out
Closing Balance    = Opening Balance + Net Movement - Assigned - Expended
Opening Balance    = closing balance of all records dated before startDate (0 when no startDate)
```

The dashboard (`GET /api/assets/dashboard`) runs one set of grouped sums for the records *before* the start date, which gives the opening balance. It runs a second set for the records *inside* the date range, which gives the period movement. Each set is grouped by equipment type.

Stock integrity

Transfers, assignments and expenditures go through `withStockCheck` . It works out the base's current balance for that equipment type and inserts the record inside the **same Serializable transaction**. If the quantity is more than the balance, the request is rejected with **400 Insufficient stock**. If another request changes the stock at the same time, PostgreSQL aborts one of them and the API returns **409**, asking the client to retry.

Auditing

Every `POST/PUT/PATCH/DELETE` request is written to the `AuditLog` table after the response is sent. Each entry records the user, method, path, status code, IP address, request body and duration. Any field whose name contains "password" is stored as `[REDACTED]` .

Technology stack

Layer	Technology	Notes
Frontend	React 19, React Router 7, Vite 8	Single-page app. Dev server on :5173 forwards /api to the backend
Backend	Node.js (22.x tested), Express 5	REST/JSON API on :5000. Errors thrown in async handlers reach the error middleware
Auth	jsonwebtoken, bcryptjs	HS256 JWT (default expiry 1d). Passwords hashed with bcrypt (10 rounds)
ORM	Prisma 6	Schema in backend/models/schema.prisma. Migrations and seed script live alongside it
Database	PostgreSQL 17 (Docker)	Container military_assets_db, host port 5435

2 Overall architecture

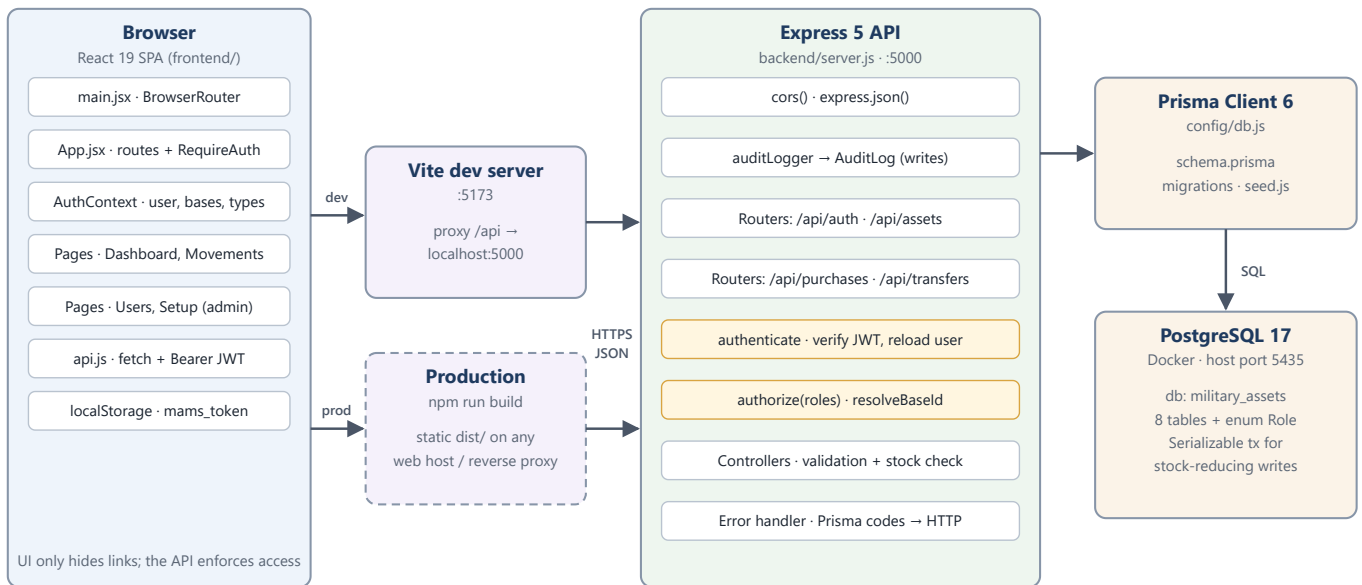


Figure 1: Component architecture. The highlighted middleware handles authentication and authorization.

Request lifecycle (for example POST /api/transfers)

- 1 Client**
api.js reads the JWT from localStorage, adds `Authorization: Bearer <token>` and sends the JSON body to `/api/transfers`.
- 2 Global middleware**
`cors()` runs first, then `express.json()` parses the body. `auditLogger` registers a hook that writes the AuditLog row once the response is sent.
- 3 authenticate**
Verifies the token signature and expiry, then reloads the user from the database on every request. Role or base changes and deleted users take effect immediately. Returns 401 on failure.
- 4 authorize / resolveBaseId**
The route-level `authorize(...roles)` returns 403 for roles not on the list. Inside the controller, `resolveBaseId` locks non-admin users to their own base and returns 403 if they request a different one.
- 5 Controller**
Validates IDs, quantities and dates. For stock-reducing writes it opens a Serializable transaction, sums the movements and inserts the record.
- 6 Error handler**
Converts errors to HTTP responses: Prisma `P2002→409`, `P2003→400`, `P2025→404`, `httpError(status)` keeps its status, malformed JSON → 400, anything else → 500.

Project structure

```
backend/  
├ server.js          app + error handler  
├ docker-compose.yml PostgreSQL :5435  
├ config/db.js       PrismaClient  
├ routes/            4 routers  
├ controllers/       handlers, balances  
├ middlewares/  
│ ├── authMiddleware  JWT → req.user  
│ ├── rbacMiddleware  authorize, scope  
│ └── auditMiddleware  AuditLog writer  
└ models/  
  ├── schema.prisma  
  ├── migrations/    ..._init  
  └ seed.js
```

```
frontend/  
├ vite.config.js     /api proxy  
├ index.html  
├ dist/             build output  
└ src/  
  ├── main.jsx       router + auth  
  ├── App.jsx        routes, guards  
  ├── api.js         fetch + token  
  ├── context/       AuthContext  
  ├── components/    Layout, Filters  
  └ pages/  
    ├── Login, Dashboard  
    ├── MovementPage (+ configs)  
    └ Users, Setup (admin)
```

3 ER schema diagram

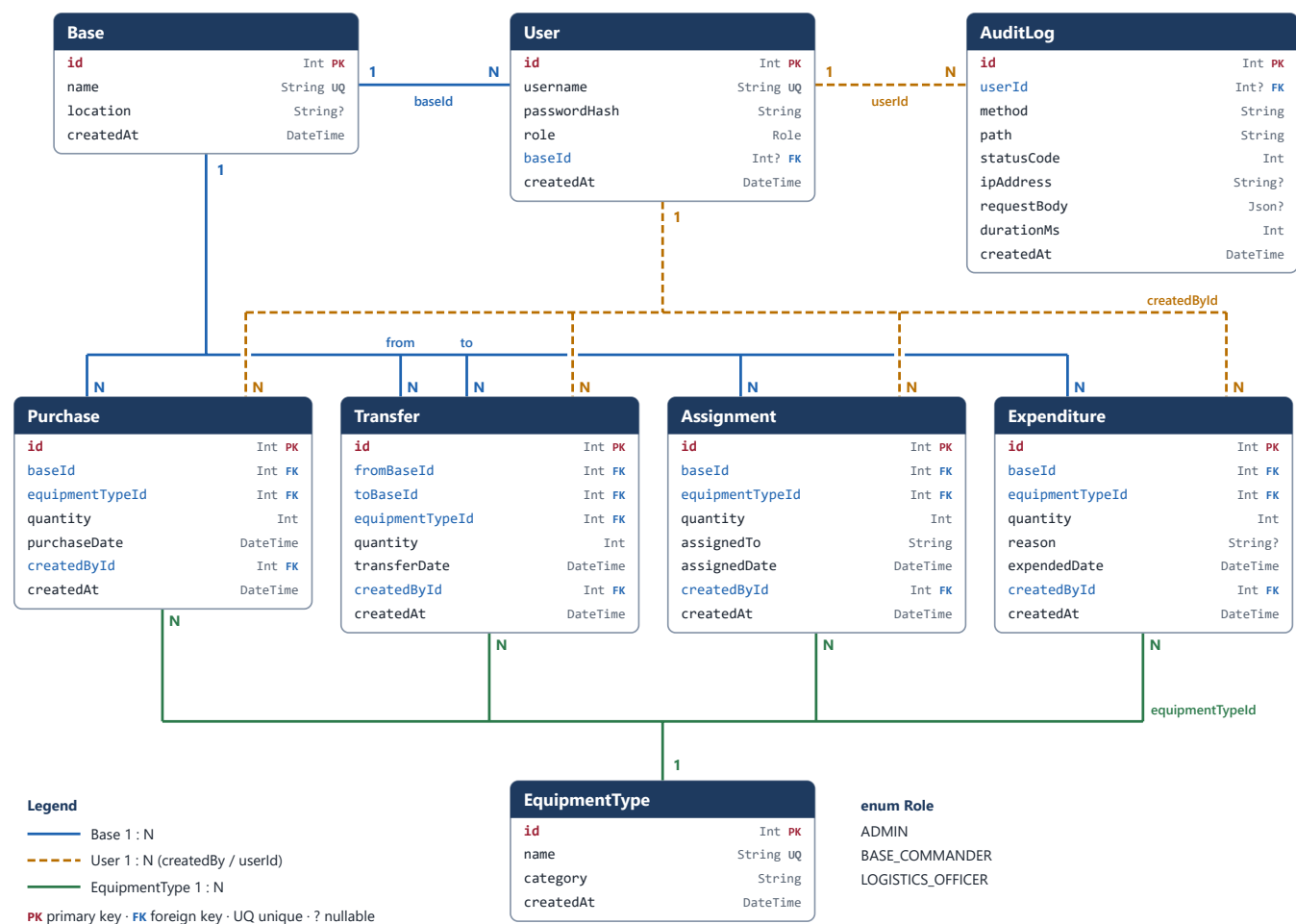


Figure 2: Entity relationship diagram (PostgreSQL, generated from schema.prisma)

Relationships and referential actions

Parent	Child	FK column	Cardinality	On delete
Base	User	baseId	1 : N (optional)	SET NULL
Base	Purchase / Assignment / Expenditure	baseId	1 : N	RESTRICT
Base	Transfer	fromBaseId , toBaseId	1 : N (two relations)	RESTRICT
EquipmentType	Purchase / Transfer / Assignment / Expenditure	equipmentTypeId	1 : N	RESTRICT
User	Purchase / Transfer / Assignment / Expenditure	createdById	1 : N	RESTRICT
User	AuditLog	userId	1 : N (optional)	SET NULL

Constraints and indexes

Table	Unique	Indexes (for balance queries)
Base	name	—
User	username	—

Table	Unique	Indexes (for balance queries)
EquipmentType	name	—
Purchase	—	(baseId, equipmentTypeId, purchaseDate)
Transfer	—	(fromBaseId, equipmentTypeId, transferDate) (toBaseId, equipmentTypeId, transferDate)
Assignment	—	(baseId, equipmentTypeId, assignedDate)
Expenditure	—	(baseId, equipmentTypeId, expendedDate)
AuditLog	—	(userId, createdAt)

Business rule enforced in `authController.createUser`: an **ADMIN** must have `baseId = null`, and **BASE_COMMANDER** and **LOGISTICS_OFFICER** must have a `baseId`. Quantities must be positive integers (checked in the API, not by a database constraint).

4 API endpoint reference

Base URL: `http://localhost:5000/api`. Every endpoint except login needs `Authorization: Bearer <JWT>`. Request and response bodies are JSON. Dates are ISO strings. A date-only `endDate` (for example `2026-09-16`) includes the whole day.

4.1 Authentication and users: `/api/auth`

Method	Path	Roles	Request	Response / errors
POST	<code>/auth/login</code>	Public	<code>{ username, password }</code>	200 <code>{ token, user: {id,username,role,baseId} }</code> 400 missing fields · 401 invalid credentials
GET	<code>/auth/me</code>	All roles	—	200 <code>{ user: {id,username,role,baseId} }</code> · 401
GET	<code>/auth/users</code>	ADMIN	—	200 users with <code>base{id,name}</code> , ordered by id · 403
POST	<code>/auth/users</code>	ADMIN	<code>{ username, password, role, baseId }</code> username ≥ 3 chars, password ≥ 8 chars; baseId null for ADMIN, required otherwise	201 created user · 400 validation · 409 username already exists

4.2 Assets, dashboard and reference data: `/api/assets`

Method	Path	Roles	Request	Response / errors
GET	<code>/assets/dashboard</code>	All roles	Query: <code>baseId?</code> <code>equipmentTypeId?</code> <code>startDate? endDate?</code>	200 <code>{ filters, totals, items[] }</code> per equipment type · 400 bad date/range · 403 other base
GET	<code>/assets/bases</code>	All roles	—	200 every base (needed to pick a transfer destination)
POST	<code>/assets/bases</code>	ADMIN	<code>{ name, location? }</code>	201 · 400 name required · 409 duplicate name
GET	<code>/assets/equipment-types</code>	All roles	—	200 all types ordered by name
POST	<code>/assets/equipment-types</code>	ADMIN	<code>{ name, category }</code>	201 · 400 · 409 duplicate name
GET	<code>/assets/assignments</code>	ADMIN BASE_COMMANDER	Query: <code>baseId?</code> <code>equipmentTypeId?</code> <code>startDate? endDate?</code>	200 list (newest first) including base, equipmentType, createdBy
POST	<code>/assets/assignments</code>	ADMIN BASE_COMMANDER	<code>{ baseId*, equipmentTypeId, quantity, assignedTo, assignedDate? }</code>	201 · 400 validation / insufficient stock · 403 · 409 concurrent change
GET	<code>/assets/expenditures</code>	ADMIN BASE_COMMANDER	Query: <code>baseId?</code> <code>equipmentTypeId?</code> <code>startDate? endDate?</code>	200 list (newest first)

Method	Path	Roles	Request	Response / errors
POST	/assets/expenditures	ADMIN BASE_COMMANDER	{ baseId*, equipmentTypeId, quantity, reason?, expendedDate? }	201 · 400 validation / insufficient stock · 403 · 409

4.3 Purchases: /api/purchases

Method	Path	Roles	Request	Response / errors
GET	/purchases	All roles	Query: baseId? equipmentTypeId? startDate? endDate?	200 list (newest first) including base, equipmentType, createdBy
POST	/purchases	All roles	{ baseId*, equipmentTypeId, quantity, purchaseDate? }	201 · 400 validation · 400 related record not found · 403 other base

4.4 Transfers: /api/transfers

Method	Path	Roles	Request	Response / errors
GET	/transfers	All roles	Query: baseId? equipmentTypeId? startDate? endDate?	200 list. The base filter matches transfers where the base is the source or the destination
POST	/transfers	All roles	{ fromBaseId*, toBaseId, equipmentTypeId, quantity, transferDate? }	201 · 400 same base / insufficient stock at source · 403 source is another base · 409

* **Base fields:** an ADMIN must send baseId / fromBaseId . Other roles can leave it out, and the API uses their own base. If they send a different base, the API returns **403**. The same rule applies to the baseId query filter on list and dashboard endpoints. When an admin leaves it out, the results cover all bases.

Examples

```
POST /api/auth/login
{ "username": "commander_alpha", "password": "Password@123" }

200 OK
{ "token": "eyJhbGciOi...", "user": { "id": 2, "username": "commander_alpha", "role": "BASE_COMMANDER",
"baseId": 1 } }
```

```
GET /api/assets/dashboard?startDate=2026-02-01&endDate=2026-02-28

200 OK
{
  "filters": { "baseId": 1, "equipmentTypeId": null, "startDate": "2026-02-01T00:00:00.000Z", "endDate":
"2026-02-28T23:59:59.999Z" },
  "totals": { "openingBalance": 80, "purchases": 50, "transfersIn": 0, "transfersOut": 0,
"netMovement": 50, "assigned": 30, "expended": 5, "closingBalance": 95 },
  "items": [ { "equipmentType": { "id": 1, "name": "Assault Rifle", "category": "WEAPON" },
"openingBalance": 80, "purchases": 50, "transfersIn": 0, "transfersOut": 0,
"netMovement": 50, "assigned": 30, "expended": 5, "closingBalance": 95 } ]
}
```

```
POST /api/transfers          (as logistics_officer_alpha, base 1)
{ "toBaseId": 2, "equipmentTypeId": 1, "quantity": 500 }

400 Bad Request
{ "message": "Insufficient stock: 95 available, 500 requested" }
```

Common error responses

Status	When	Body
400	Validation failure, malformed JSON, foreign key not found (P2003), insufficient stock, same-base transfer	{ "message": "..." }
401	Missing or invalid Bearer token, expired token, user deleted, wrong credentials at login	{ "message": "Authentication required" }
403	Role not allowed on the route, or a non-admin targeting another base	{ "message": "You do not have permission to perform this action" }
404	Unknown route, or record not found (P2025)	{ "message": "Route not found: GET /api/x" }
409	Unique constraint violation (P2002), or a Serializable transaction conflict (P2034)	{ "message": "Record already exists" }
500	Unexpected server error (details logged on the server only)	{ "message": "Internal Server Error" }

5 RBAC authorization matrix

Access control has two layers. **Role checks** (`authorize` on the route) decide who may call an endpoint. **Data scoping** (`resolveBaseId` in the controller) limits base-bound roles to their own base. The frontend hides links and routes to match, but the API is what actually enforces access.

5.1 API permissions

Endpoint / action	ADMIN	BASE_COMMANDER	LOGISTICS_OFFICER
POST /auth/login	✓	✓	✓
GET /auth/me	✓	✓	✓
GET /auth/users: list users	✓	✗	✗
POST /auth/users: create user	✓	✗	✗
GET /assets/dashboard: balances	✓ all / any base	✓ own base	✓ own base
GET /assets/bases	✓	✓ all names	✓ all names
POST /assets/bases	✓	✗	✗
GET /assets/equipment-types	✓	✓	✓
POST /assets/equipment-types	✓	✗	✗
GET /purchases	✓ all / any base	✓ own base	✓ own base
POST /purchases	✓ any base	✓ own base	✓ own base
GET /transfers	✓ all / any base	✓ own base (in + out)	✓ own base (in + out)
POST /transfers	✓ any source	✓ own base as source	✓ own base as source
GET /assets/assignments	✓ all / any base	✓ own base	✗
POST /assets/assignments	✓ any base	✓ own base	✗
GET /assets/expenditures	✓ all / any base	✓ own base	✗
POST /assets/expenditures	✓ any base	✓ own base	✗

✓ allowed · ✓ own base allowed, limited to the user's assigned base (a different base gives 403) · ✗ denied with 403

5.2 Frontend page access

Page (route)	ADMIN	BASE_COMMANDER	LOGISTICS_OFFICER	Notes
Login (/login)	✓	✓	✓	Public
Dashboard (/)	✓	✓	✓	Only admins see the base filter
Purchases (/purchases)	✓	✓	✓	Only admins pick the base
Transfers (/transfers)	✓	✓	✓	Only admins pick the source base
Assignments (/assignments)	✓	✓	✗	Guarded by RequireAuth(roles)
Expenditures (/expenditures)	✓	✓	✗	Guarded by RequireAuth(roles)
Users (/users)	✓	✗	✗	Admin only
Bases & Equipment (/setup)	✓	✗	✗	Admin only

5.3 Role summary

Role	Base binding	Responsibilities
ADMIN	None (baseId = null)	Full access to every base. Manages users, bases and equipment types. Can record any movement for any base
BASE_COMMANDER	Exactly one base	Oversees their own base: views balances, records purchases and transfers out, assigns equipment to personnel, records expenditures
LOGISTICS_OFFICER	Exactly one base	Handles logistics for their own base: views balances, records purchases and transfers out. Cannot assign or expend equipment

6 Setup steps

Prerequisites

Tool	Version	Purpose
Node.js + npm	20 or later (22.12 tested)	Runs the backend, the frontend tooling and Prisma
Docker Desktop	Current	Runs PostgreSQL 17 using backend/docker-compose.yml
Git Bash / PowerShell	—	Commands below work in both (Windows paths shown)

If you already have PostgreSQL installed locally, you can skip Docker and point `DATABASE_URL` at your own server.

Backend

1 Open the backend folder

```
cd "Military Asset Management System\backend"
```

2 Create the environment file

Copy the example and set a long random `JWT_SECRET`.

```
copy .env.example .env          # PowerShell/cmd (Git Bash: cp .env.example .env)

PORT=5000
NODE_ENV=development
DATABASE_URL="postgresql://military:military@localhost:5435/military_assets?schema=public"
JWT_SECRET=replace-with-a-long-random-string
JWT_EXPIRES_IN=1d
# optional: SEED_PASSWORD=YourDevPassword
```

3 Start PostgreSQL

```
docker compose up -d
docker ps          # military_assets_db should be "healthy"
```

4 Install dependencies

```
npm install
```

5 Apply migrations and generate the Prisma client

```
npx prisma migrate dev          # applies models/migrations, runs prisma generate
```

6 Seed the sample data

Creates 2 bases, 3 equipment types and 5 users. The seed uses upserts, so running it again is safe.

```
npm run seed                    # = prisma db seed → node models/seed.js
```

7 Start the API

```
npm run dev                # nodemon, http://localhost:5000
# or: npm start
```

Frontend

1 Install and run the dev server

```
cd "Military Asset Management System\frontend"
npm install
npm run dev                # http://localhost:5173 (/api is proxied to :5000)
```

2 Production build (optional)

```
npm run build              # writes dist/index.html + dist/assets/*.js, *.css
npm run preview            # serve dist/ locally to check the build
```

In production, the web server that hosts `dist/` must forward `/api` to the backend.

Useful maintenance commands (backend folder)

Command	What it does
<code>npx prisma migrate reset</code>	Drops the database, re-applies all migrations and re-runs the seed (all data is lost)
<code>npx prisma studio</code>	Opens a browser GUI to view and edit tables
<code>npx prisma generate</code>	Regenerates the Prisma client after schema changes
<code>npx prisma migrate dev --name <change></code>	Creates and applies a new migration from schema.prisma changes
<code>docker compose down -v</code>	Stops PostgreSQL and deletes its data volume

7 Sample test accounts

Created by `backend/models/seed.js` . Every account uses the same password, `Password@123` , unless `SEED_PASSWORD` is set in `.env` .

Development only. Do not run the seed against production, or change these passwords right away. The seed prints the password to the console.

Username	Password	Role	Base	Use it to test
<code>admin_user</code>	<code>Password@123</code>	ADMIN	— (all bases)	User management, bases/equipment setup, cross-base reporting and movements
<code>commander_alpha</code>	<code>Password@123</code>	BASE_COMMANDER	Alpha Base (North Sector)	Assignments, expenditures, and 403 when requesting Delta data
<code>logistics_officer_alpha</code>	<code>Password@123</code>	LOGISTICS_OFFICER	Alpha Base (North Sector)	Purchases and transfers out. Assignments/expenditures are blocked
<code>commander_delta</code>	<code>Password@123</code>	BASE_COMMANDER	Delta Base (South Sector)	Receiving transfers, own-base dashboard
<code>logistics_officer_delta</code>	<code>Password@123</code>	LOGISTICS_OFFICER	Delta Base (South Sector)	Own-base purchases and transfers

Seeded reference data

Base	Location	Equipment type	Category
Alpha Base	North Sector	Assault Rifle	WEAPON
		Armored Vehicle	VEHICLE
Delta Base	South Sector	5.56mm Ammunition	AMMUNITION

The seed creates no stock movements, so every balance starts at 0. Record a purchase first, otherwise transfers, assignments and expenditures fail with "Insufficient stock".

Suggested test walkthrough

#	Log in as	Action	Expected result
1	<code>logistics_officer_alpha</code>	Purchase 100 × Assault Rifle	201. Alpha dashboard closing balance = 100
2	<code>logistics_officer_alpha</code>	Transfer 20 × Assault Rifle to Delta Base	201. Alpha shows transfers out 20, closing balance 80
3	<code>logistics_officer_alpha</code>	Transfer 500 × Assault Rifle to Delta Base	400 "Insufficient stock: 80 available, 500 requested"
4	<code>logistics_officer_alpha</code>	Open <code>/assignments</code> , or call <code>GET /api/assets/assignments</code>	UI redirects to the dashboard. API returns 403
5	<code>commander_alpha</code>	Assign 30 × Assault Rifle to "Sgt. Rao"	201. Closing balance 50

#	Log in as	Action	Expected result
6	commander_alpha	Record expenditure 5 × Assault Rifle, reason "Training"	201. Closing balance 45
7	commander_alpha	GET /api/purchases?baseId=2 (Delta)	403 "You can only access data for your assigned base"
8	commander_delta	Dashboard	Transfers in 20, closing balance 20
9	admin_user	Dashboard with no base filter	Both bases combined. Transfers in and out cancel out
10	admin_user	Create user with role BASE_COMMANDER and no base	400 "baseId is required for this role"